



---

# Python Schnelleinstieg Add Ons

**Michael Inden**

**Freiberuflicher Trainer und Autor**



**<https://github.com/Michaeli71/Python-Workshop>**

---

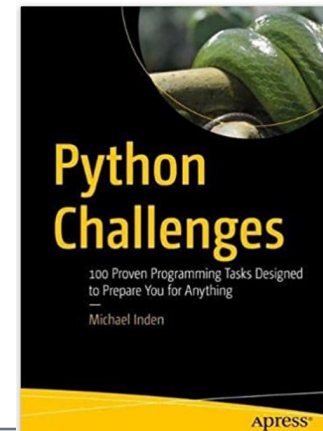
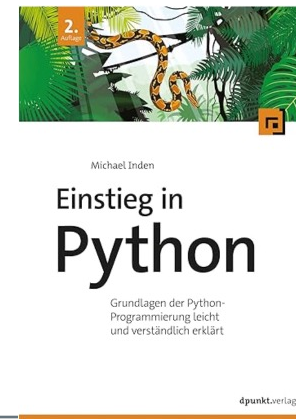
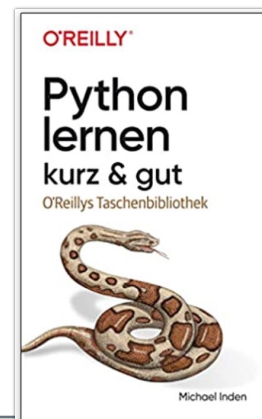
# Speaker Intro



- **Michael Inden**, Jahrgang 1971
- Diplom-Informatiker, C.v.O. Uni Oldenburg
- ~8 ¼ Jahre **SSE** bei Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Jahre **TPL, SA** bei IVU Traffic Technologies AG in Aachen
- ~4 ¼ Jahre **LSA / Trainer** bei Zühlke Engineering AG in Zürich
- ~3 Jahre **TL / CTO** bei Direct Mail Informatics / ASMIQ in Zürich
- ~3 ½ Jahre **Head of Development** bei Adcubum in Zürich
- Freiberuflicher **Consultant, Trainer** und **Konferenz-Speaker**
- Autor und Gutachter bei dpunkt.verlag, O'Reilly und APress

E-Mail: [michael\\_inden@hotmail.com](mailto:michael_inden@hotmail.com)

Kurse: **Bitte spricht mich an!**



<https://github.com/Michaeli71/Python-Workshop>



---

# Agenda

---

# Workshop Contents

---



- **Add On 1: Zeitmessung**
- **Add On 2: Decorators**
- **Weitere auf Wunsch**





---

# Add On 1: Zeitmessung

- Grundsätzliches
  - Modul time
  - Modul timeit
  - Modul cProfile
-



- **Manchmal ist die Performance eines Programms nicht zufriedenstellend**
  - **Das kann unterschiedlichste Ursachen haben: Architektur, Design, Algorithmen, ungeschickte Aufrufe, tief verschachtelte Schleifen usw.**
  - **Wesentlich: Messen, Messen, Messen**
  - **Aber wie macht man das und bestimmt man die kritischen Teile?**
  - **Erst nach Analysen und Nachdenken mit Optimierungen anfangen**
  - **Danach unbedingt nochmals messen und prüfen, ob gewünschte Effekte eingetreten sind**
-



- **Idee: Stoppuhr-Messung: Start und Ende bestimmen**

```
import time

start = time.time()
for i in range(10_000):
    print("Hard Work")
    i = i ** 2

end = time.time()

print("% s seconds" % (end - start))
```

```
...
Hard Work
Hard Work
Hard Work
Hard Work
0.007323741912841797 seconds
```



- **Idee: Stoppuhr-Messung: Start und Ende bestimmen**

```
import time
```

```
def fibonacci(n):  
    if n <= 1:  
        return n  
  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
start_time = time.time()  
result = fibonacci(42)  
end_time = time.time()
```

```
print(f"Fibonacci(42) = {result}")  
print(f"Time taken: {end_time - start_time:.4f} seconds")
```

```
Fibonacci(42) = 267914296  
Time taken: 17.9224 seconds
```



- **Idee: Stoppuhr-Messung: Start und Ende bestimmen**

```
import time
```

```
number = 100_000
```

```
start = time.time_ns()
for i in range(number):
    nums = [13, 11, 7, 5, 3, 2]
    list(reversed(nums))
end = time.time_ns()
```

```
start2 = time.time_ns()
for i in range(number):
    nums = [13, 11, 7, 5, 3, 2]
    nums[::-1]
end2 = time.time_ns()
```

```
print(f"Using reversed() fn.: {(end-start) / number} ns")
```

```
print(f"Using list slicing: {(end2-start2) / number} ns")
```

Using reversed() fn.: 149.14 ns

Using list slicing: 91.54 ns



- **Verbesserte Messung, ohne Mix mit dem Programmcode**

```
import timeit
```

```
setup = "nums = [13, 11, 7, 5, 3, 2]"
```

```
number = 100_000
```

```
stmt1 = "list(reversed(nums))"
```

```
stmt2 = "nums[::-1]"
```

```
t1 = timeit.timeit(setup=setup, stmt=stmt1, number=number)
```

```
t2 = timeit.timeit(setup=setup, stmt=stmt2, number=number)
```

```
print(f"Using reversed() fn.: {t1}")
```

```
print(f"Using list slicing: {t2}")
```

```
Using reversed() fn.: 0.00847270899976138  
Using list slicing: 0.00416150000091875
```



- **Verbesserte Messung, minimale Zeit und Einfluss von Repeat + Count**

```
def my_function():  
    a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
    b = [i**2 for i in a]  
    return b
```

```
def my_function2():  
    for i in range(100):  
        a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
        b = a * i  
    return b
```

```
count = 1000  
execution_time = min(timeit.repeat(my_function, repeat=100, number=count))  
print(f"Execution time: {execution_time:.7f} seconds")  
print(f"Single Execution time: {execution_time/count:.7f} seconds")
```

```
execution_time2 = min(timeit.repeat(my_function2, repeat=100, number=count))  
print(f"Execution time2: {execution_time2:.7f} seconds")  
print(f"Single Execution time: {execution_time2/count:.7f} seconds")
```

Execution time: 0.0002283 seconds  
Single Execution time: 0.0000002 seconds  
Execution time2: 0.0375617 seconds  
Single Execution time: 0.0000376 seconds



- **Gegeben seien folgende Funktionen mit willkürlich gewählten Berechnungen:**

```
def sample_func(i):  
    for j in range(i * 100):  
        random.randint(2, 100)  
        if math.pow(j, 2) > 100:  
            random.choice(["a", "b", "c"])  
        a = [1] * (10**4) + [2] * (10**5)  
    return 1
```

```
def math_func(i):  
    for j in range(i * 2000):  
        math.sqrt(j)  
    return math.sin(i) + math.cos(i)
```

```
def str_func(i):  
    result = ""  
    for j in range(i * 5000):  
        result += str(j)  
    return i
```



- **Detaillierte Analyse**

```
def my_function():  
    result = 0  
    for i in range(100):  
        result += math_func(i)  
        result += sample_func(i)  
        result += str_func(i)
```

```
    print("result:", result)
```

```
profiler = cProfile.Profile()  
profiler.enable()  
# beliebige Aufrufe zum Messen  
my_function()  
# Disable the profiler and print stats  
profiler.disable()
```

```
s = io.StringIO()  
stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")  
stats.print_stats(20) # Print top 20 functions  
print(s.getvalue())
```

```
# nur eine Funktion ausführen  
profiler = cProfile.Profile()  
profiler.run('my_function()')
```

# Modul cProfile



result: 5049.98458714693  
16638528 function calls in 76.682 seconds

Ordered by: cumulative time

| ncalls  | tottime | percall | cumtime | percall | filename:lineno(function)                                          |
|---------|---------|---------|---------|---------|--------------------------------------------------------------------|
| 1       | 0.000   | 0.000   | 76.682  | 76.682  | {built-in method builtins.exec}                                    |
| 1       | 0.000   | 0.000   | 76.682  | 76.682  | <string>:1(<module>)                                               |
| 1       | 0.010   | 0.010   | 76.682  | 76.682  | /../cProfile_Example2.py:30(my_function)                           |
| 100     | 72.360  | 0.724   | 73.114  | 0.731   | /../cProfile_Example2.py:8(sample_func)                            |
| 100     | 2.480   | 0.025   | 2.480   | 0.025   | /../cProfile_Example2.py:23(str_func)                              |
| 100     | 0.636   | 0.006   | 1.079   | 0.011   | /../cProfile_Example2.py:17(math_func)                             |
| 990000  | 0.442   | 0.000   | 0.442   | 0.000   | {built-in method math.sqrt}                                        |
| 495000  | 0.142   | 0.000   | 0.360   | 0.000   | /../3.14/lib/python3.14/random.py:335(randint)                     |
| 988911  | 0.228   | 0.000   | 0.347   | 0.000   | /../3.14/lib/python3.14/random.py:245(_randbelow_with_getrandbits) |
| 493911  | 0.132   | 0.000   | 0.346   | 0.000   | /../3.14/lib/python3.14/random.py:347(choice)                      |
| 1298468 | 0.071   | 0.000   | 0.071   | 0.000   | {method 'getrandbits' of '_random.Random' objects}                 |
| 988911  | 0.048   | 0.000   | 0.048   | 0.000   | {method 'bit_length' of 'int' objects}                             |
| 495000  | 0.047   | 0.000   | 0.047   | 0.000   | {built-in method math.pow}                                         |
| 987822  | 0.045   | 0.000   | 0.045   | 0.000   | {built-in method builtins.len}                                     |
| 990000  | 0.041   | 0.000   | 0.041   | 0.000   | {built-in method _operator.index}                                  |
| 100     | 0.000   | 0.000   | 0.000   | 0.000   | {built-in method math.sin}                                         |
| 100     | 0.000   | 0.000   | 0.000   | 0.000   | {built-in method math.cos}                                         |
| 1       | 0.000   | 0.000   | 0.000   | 0.000   | {built-in method builtins.print}                                   |
| 1       | 0.000   | 0.000   | 0.000   | 0.000   | {method 'disable' of '_lsprof.Profiler' objects}                   |

# Modul cProfile



```
def slow_remove_names_from_list(names, names_to_remove):
    result = []
    while names:
        name = names.pop()
        if not should_remove(name, names_to_remove):
            result.append(name)

    return result
```

```
def should_remove(name, names_to_remove):
    for name_to_remove in names_to_remove:
        if name.lower() == name_to_remove.lower():
            return True
    return False
```

```
source_names = ["Tim", "Peter", "Mike", "Maria", "michael",
                "mike", "Tom", "Sophie", "Anne", "James"] * 20_000
names_to_remove = ["Tim", "Peter", "Mike", "Jim", "John",
                  "Sophia", "Janne", "Timo"] * 1000
```

```
slow_remove_names_from_list(source_names, names_to_remove)
```



- **Analyse: Ausgangslage katastrophal**

1920880004 function calls in **214.929** seconds

Ordered by: standard name

| ncalls            | tottime        | percall      | cumtime        | percall      | filename:lineno(function)                        |
|-------------------|----------------|--------------|----------------|--------------|--------------------------------------------------|
| 1                 | 0.000          | 0.000        | 214.928        | 214.928      | <string>:1(<module>)                             |
| 1                 | 0.049          | 0.049        | 214.928        | 214.928      | starting_point.py:1(slow_remove_names_from_list) |
| <b>200000</b>     | <b>120.823</b> | <b>0.001</b> | <b>214.857</b> | <b>0.001</b> | <b>starting_point.py:11(should_remove)</b>       |
| 1                 | 0.000          | 0.000        | 214.928        | 214.928      | {built-in method builtins.exec}                  |
| 120000            | 0.008          | 0.000        | 0.008          | 0.000        | {method 'append' of 'list' objects}              |
| 1                 | 0.000          | 0.000        | 0.000          | 0.000        | {method 'disable' of '_lsprof.Profiler' objects} |
| <b>1920360000</b> | <b>94.034</b>  | <b>0.000</b> | <b>94.034</b>  | <b>0.000</b> | <b>{method 'lower' of 'str' objects}</b>         |
| 200000            | 0.015          | 0.000        | 0.015          | 0.000        | {method 'pop' of 'list' objects}                 |



- **Verbesserung: massiv schneller, aber immer noch ziemlich langsam**

728004 function calls in **9.769** seconds

Ordered by: standard name

| ncalls        | tottime      | percall      | cumtime      | percall      | filename:lineno(function)                           |
|---------------|--------------|--------------|--------------|--------------|-----------------------------------------------------|
| 1             | 0.001        | 0.001        | 9.769        | 9.769        | <string>:1(<module>)                                |
| 1             | 0.063        | 0.063        | 9.769        | 9.769        | optimized_version.py:1(fast_remove_names_from_list) |
| <b>200000</b> | <b>9.673</b> | <b>0.000</b> | <b>9.673</b> | <b>0.000</b> | <b>optimized_version.py:17(should_remove)</b>       |
| 1             | 0.000        | 0.000        | 9.769        | 9.769        | {built-in method builtins.exec}                     |
| 120000        | 0.008        | 0.000        | 0.008        | 0.000        | {method 'append' of 'list' objects}                 |
| 1             | 0.000        | 0.000        | 0.000        | 0.000        | {method 'disable' of '_lsprof.Profiler' objects}    |
| <b>208000</b> | <b>0.011</b> | <b>0.000</b> | <b>0.011</b> | <b>0.000</b> | <b>{method 'lower' of 'str' objects}</b>            |
| 200000        | 0.013        | 0.000        | 0.013        | 0.000        | {method 'pop' of 'list' objects}                    |



- **Verbesserung 2: rasant 😊**

720012 function calls in **0.103** seconds

Ordered by: standard name

| ncalls        | tottime      | percall      | cumtime      | percall      | filename:lineno(function)                             |
|---------------|--------------|--------------|--------------|--------------|-------------------------------------------------------|
| 1             | 0.001        | 0.001        | 0.103        | 0.103        | <string>:1(<module>)                                  |
| 1             | 0.050        | 0.050        | 0.103        | 0.103        | optimized_version_2.py:1(fast_remove_names_from_list) |
| <b>200000</b> | <b>0.026</b> | <b>0.000</b> | <b>0.026</b> | <b>0.000</b> | <b>optimized_version_2.py:20(should_remove)</b>       |
| 1             | 0.000        | 0.000        | 0.103        | 0.103        | {built-in method builtins.exec}                       |
| 120000        | 0.006        | 0.000        | 0.006        | 0.000        | {method 'append' of 'list' objects}                   |
| 1             | 0.000        | 0.000        | 0.000        | 0.000        | {method 'disable' of '_lsprof.Profiler' objects}      |
| <b>200008</b> | <b>0.011</b> | <b>0.000</b> | <b>0.011</b> | <b>0.000</b> | <b>{method 'lower' of 'str' objects}</b>              |
| 200000        | 0.009        | 0.000        | 0.009        | 0.000        | {method 'pop' of 'list' objects}                      |



---

# Add On 2: Decorators

- Grundsätzliches
  - Decorator-Syntax
  - Real-World-Usage
-

## Recap: Besonderheit: Verschachtelte Funktionen definieren



```
def f():  
    print("f1")  
  
    def g():  
        print("g!")  
  
        # (noch) nicht sichtbar!  
        # h()  
        def h():  
            print("h!")  
  
            h()  
  
        # nicht (mehr) sichtbar!  
        # h()  
        g()  
        print("f2")  
  
f()  
print("nach f")
```

```
f1  
g!  
h!  
f2  
nach f
```



- **Alles in Python ist ein Objekt, auch Funktionen**

```
def inner_function():  
    print("inner_function is called")
```

```
def outer_function(func):  
    print("outer_function is called")  
    func()
```

```
# Funktionen an Funktionen übergeben  
outer_function(inner_function)
```

```
outer_function is called  
inner_function is called
```



- **Alles in Python ist ein Objekt, auch innere Funktionen als Rückgabe**

```
def outer_function():  
    def inner_function():  
        print("inner_function is called")  
  
    print("outer_function is called")  
    return inner_function
```

```
result = outer_function()  
print(result)  
result()
```

```
outer_function is called  
<function outer_function.<locals>.inner_function at 0x104702fb0>  
inner_function is called
```



- **Beides kombinieren**

```
def add_one_decorator(func):  
    def add_one():  
        value = func()  
        return value + 1  
  
    return add_one
```

```
def example_function():  
    return 6
```

```
result_value = add_one_decorator(example_function)  
print(result_value())
```



- **Kurzschreibweise mit @**

```
# Kurzform des Decorators:  
# example_function = add_one_decorator(example_function)  
@add_one_decorator  
def example_function():  
    return 41
```

```
result = example_function()  
print(result)
```

42

## Decorator und Parameter: Problematik



```
def add_one_decorator(func):  
    def add_one():  
        value = func()  
        return value + 1  
  
    return add_one
```

```
@add_one_decorator  
def add(a, b):  
    return a + b
```

```
print(add(1, 2))
```

```
Traceback (most recent call last):  
  File  
"/Users/michaelinden/PycharmProjects/QuickStartAdd0ns/src/Add0n_02_Decorators/decorators_argument  
s_problem.py", line 14, in <module>  
    print(add(1, 2))  
          ~~~~  
TypeError: add_one_decorator.<locals>.add_one() takes 0 positional arguments but 2 were given
```



```
def add_one_decorator(func):  
    def add_one(*args, **kwargs):  
        value = func(*args, **kwargs)  
        return value + 1  
  
    return add_one
```

```
@add_one_decorator  
def add(a, b):  
    return a + b
```

```
print(add(39, 2))
```



---

# Real-World Usage





- Ähnlich wie Modul `timeit`

```
from time import time
```

```
def mytimeit(func):  
    def timed(*args, **kw):  
        ts = time()  
        result = func(*args, **kw)  
        te = time()  
  
        print("func:%r args:[%r, %r] took: %3.7f sec" %  
              (func.__name__, args, kw, te - ts))  
        return result  
  
    return timed
```



- Ähnlich wie Modul `timeit`

```
@mytimeit
def long_time(n):
    for i in range(n):
        for j in range(500_000):
            i * j
```

```
long_time(1000)
```

```
func:'long_time' args:[(1000,), {}] took: 6.5986309 sec
```



**Das war ja einfach ...  
alles klar soweit?**

## Usage Case: Zeitmessung ... ups, in der Art bei Rekursion ungünstig



```
@mytimeit
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10))
```

```
func:'fibonacci' args:[(1,), {}] took: 0.0000000 sec
func:'fibonacci' args:[(0,), {}] took: 0.0000000 sec
func:'fibonacci' args:[(2,), {}] took: 0.0000057 sec
func:'fibonacci' args:[(1,), {}] took: 0.0000000 sec
func:'fibonacci' args:[(3,), {}] took: 0.0000088 sec
func:'fibonacci' args:[(1,), {}] took: 0.0000000 sec
...
func:'fibonacci' args:[(6,), {}] took: 0.0000300 sec
func:'fibonacci' args:[(8,), {}] took: 0.0000789 sec
func:'fibonacci' args:[(10,), {}] took: 0.0002198 sec
55
```



**Hat jemand von euch  
eine Idee?**

## Usage Case: Trick der Indirektion bei Rekursion



```
def fibonacci(n):  
    if n <= 1:  
        return n  
  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

*# Lösung für rekursive Funktionen*

*@mytimeit*

```
def fibonacci10():  
    return fibonacci(10)
```

*# Lösung für rekursive Funktionen*

*@mytimeit*

```
def fibonacci42():  
    return fibonacci(42)
```

```
func:'fibonacci10' args:[(), {}] took: 0.0000091 sec  
55  
func:'fibonacci42' args:[(), {}] took: 20.4970813 sec  
267914296
```

## Usage Case: Logging / Tracing / Auditing



```
import logging

def function_logger(func):
    logging.basicConfig(level = logging.INFO, filename="main.log")

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        logging.info(f"{func.__name__} ran with positional arguments: {args} "
                    f"and keyword arguments: {kwargs}. Return value: {result}")
        return result

    return wrapper

@function_logger
def add_one(value):
    return value + 1

print(add_one(1))
print(add_one(41))
```

```
INFO:root:add_one ran with positional arguments: (1,) and keyword arguments: {}. Return value: 2
INFO:root:add_one ran with positional arguments: (41,) and keyword arguments: {}. Return value: 42
```

## Usage Case: Caching / Memoization mit Built-in-Decorator lru\_cache

---



```
from functools import lru_cache

@lru_cache
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10))
print(fibonacci(42))
print(fibonacci(100))
```

```
55
267914296
354224848179261915075
```

## Usage Case: Kombination von Decorators

---



```
# möglich, aber bei Rekursion nicht gut
# @mytimeit
@function_logger
@lru_cache
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
@mytimeit
def fibonacci42():
    return fibonacci(42)
```

```
@mytimeit
def fibonacci100():
    return fibonacci(100)
```

---

## Usage Case: Kombination von Decorators



```
#print(fibonacci(42))  
print(fibonacci42())  
print(fibonacci100())
```

```
func:'fibonacci42' args:[()], {} took: 0.0007131 sec  
267914296  
func:'fibonacci100' args:[()], {} took: 0.0009761 sec  
354224848179261915075
```

```
...  
INFO:root:fibonacci ran with positional arguments: (96,) and keyword arguments: {}. Return value:  
51680708854858323072  
INFO:root:fibonacci ran with positional arguments: (98,) and keyword arguments: {}. Return value:  
135301852344706746049  
INFO:root:fibonacci ran with positional arguments: (97,) and keyword arguments: {}. Return value:  
83621143489848422977  
INFO:root:fibonacci ran with positional arguments: (99,) and keyword arguments: {}. Return value:  
218922995834555169026  
INFO:root:fibonacci ran with positional arguments: (98,) and keyword arguments: {}. Return value:  
135301852344706746049  
INFO:root:fibonacci ran with positional arguments: (100,) and keyword arguments: {}. Return value:  
354224848179261915075
```

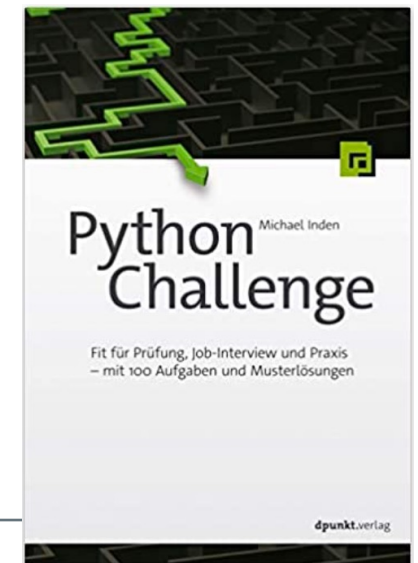
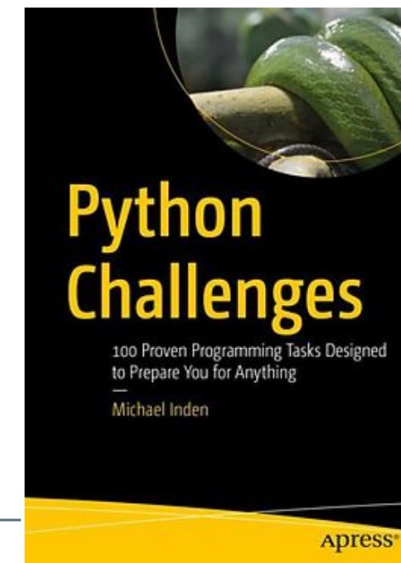
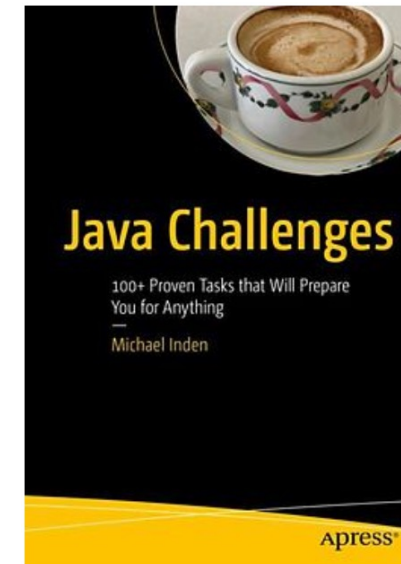
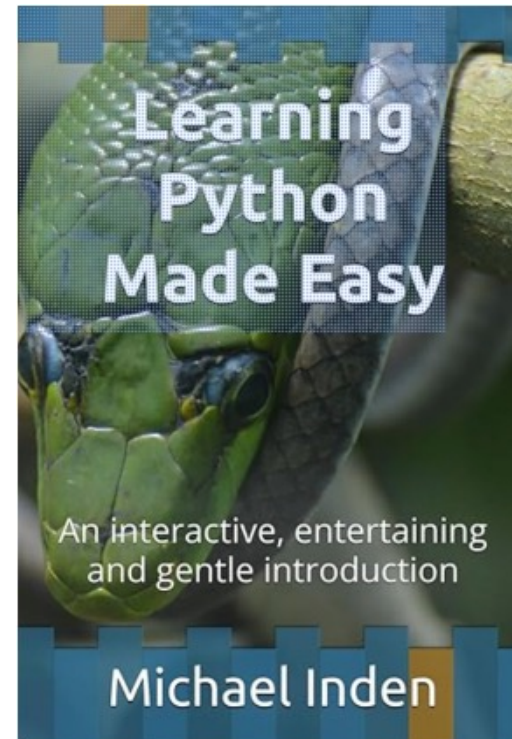


---

# Questions?

---

Help





---

# Thank You

---